PROJECT REPORT

HOMEWORK 6

Written and Prepared by:

Roxanne Lutz

University of New Mexico

November 14, 2025

COMPUTER/MATLAB PART 1

ORIGINAL PROBLEM STATEMENT

A well known equation that is used to model phase-separation of materials is given by

$$u\_t - \Delta u + (1 / \epsilon\text{^}2)\, f(u) = 0,\ \Omega = [0, 2\pi]\text{^}2.$$

where f (u) = u^3 − u. In 2D (two space dimensions), the solution will evolve into configurations that will generate transition layers connecting the values of −1 and 1 (representing pure material states in a binary mixture). Using the formal method of asymptotic expansions, it is possible to obtain the profile approximation of the so called diffusive interface for the above nonlinear PDE. The profile is found to be

$$u\_o(z) = \tanh(z).$$

Task: (A) approximate u_o(z) on [−3, 3] by using 11 equally spaced nodes and constructing Pn(x). (B)Then, you can repeat the process and use S(x), cubic splines. (C) Plot the original function, the classic interpolating polynomial and the cubic spline in the same figure. Describe your results. (D) In addition, can you extend your work to [−5, 5] - what happens in this case? (E) Can you continue to increase the number of subintervals to decrease error?

As a preliminary, we are requested to use the proved *Coef*, *Eval*, *Spline3_Coef*, and *Spline3_Eval* code to find the interpolating polynomial and degree-3 spline coefficients and then evaluate them accordingly. The code is pasted below to include for clarity.

```
%
=================================================================
  (1)COEF PROVIDED AS FUNCTION IN Coef.m FILE
%
=================================================================

function [a] = Coef(n,x,y)
%Coefficients for Interpolation
clear a;
for i=0:n
   a(i+1)=y(i+1);
end
for j=1:n
   for i=n:-1:j
       a(i+1)=(a(i+1)-a(i+1-1))/(x(i+1)-x(i+1-j));
```

```
   end
end
end

%
=================================================================
  (2)EVAL PROVIDED AS FUNCTION IN Eval.m FILE
%
=================================================================

function [value] = Eval(n,x,a,t)
%function evaluation - via interpolation
temp=a(end);
for i=n-1:-1:0
   temp=(temp)*(t-x(i+1))+a(i+1);
end
value=temp;
end

%
=================================================================
  (3)SPLINE3_COEF PROVIDED AS FUNCTION IN Spline3_Coef.m FILE
%
=================================================================

function [t,y,z] = Spline3_Coeff(t,y)
%function to compute the z values in the cublic spline process
clear h b u v z
n=length(t)-1;
for  i=1:n
   h(i)=t(i+1)-t(i);
   b(i)=(y(i+1)-y(i))/h(i);
end
   u(1)=2*(h(1)+h(2));
   v(1)=6*(b(2)-b(1));

   for i=2:n
       u(i)=2*(h(i)+h(i-1))-h(i-1)^2/u(i-1);
       v(i)=6*(b(i)-b(i-1))-h(i-1)*v(i-1)/u(i-1);
   end
```

```
    %values of z are computed from last to first
    z(n+1)=0;    %natural cubic spline right end point
    for i=n:-1:2      %typo on book ?
        z(i)=(v(i)-h(i)*z(i+1))/u(i);
    end
    z(1)=0;      % nat. cubic spline left end point
end

%
=================================================================
  (4)SPLINE3_EVAL PROVIDED AS FUNCTION IN Spline3_Eval.m FILE
%
=================================================================

function [value] = Spline3_Eval(t,y,z,x)
n=length(t)-1;
for i=n:-1:1
    if x-t(i)>=0
        break
    end
end
h=t(i+1)-t(i);
temp=z(i)/2+(x-t(i)).*(z(i+1)-z(i))/(6*h);
temp=-(h/6).*(z(i+1)+2*z(i))+(y(i+1)-y(i))/h+(x-t(i)).*(temp);
value=y(i)+(x-t(i)).*(temp);
end

%
=================================================================
  END CODE FOR COMP PROB 1
%
=================================================================
```

PART A

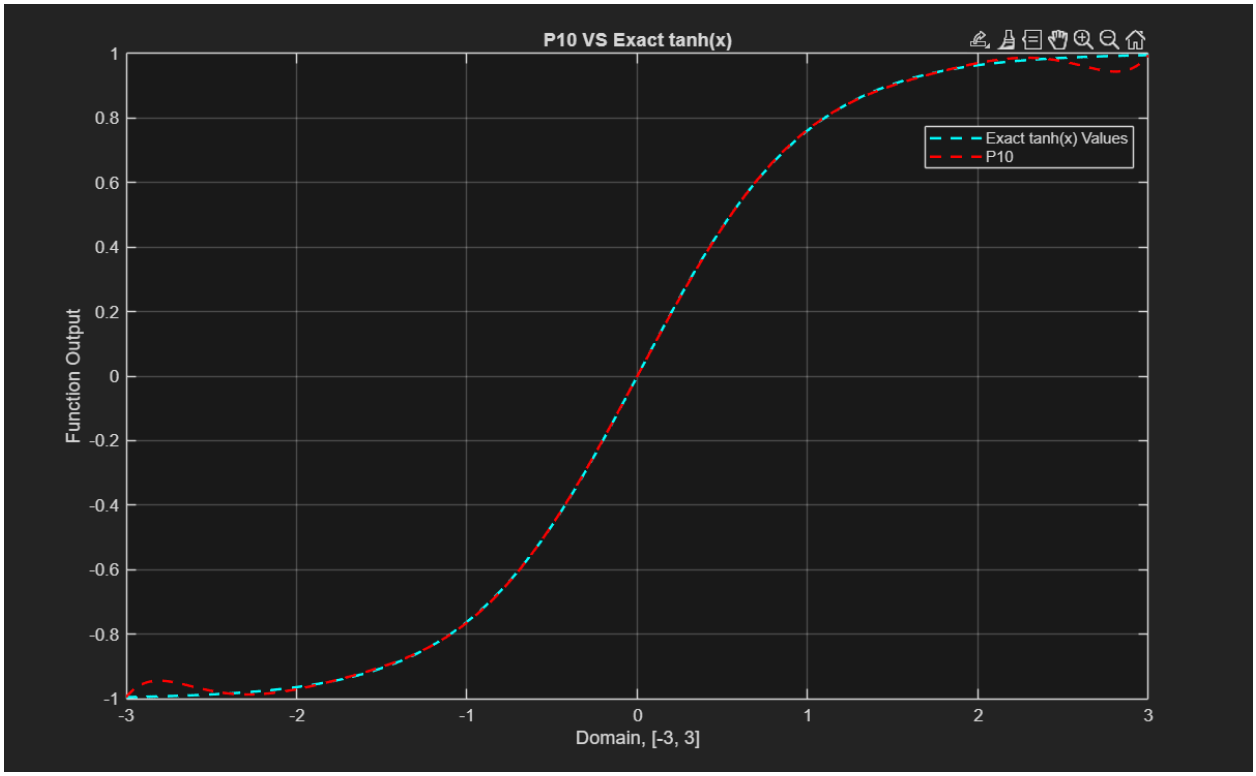
"Approximate u_o(z) on [−3, 3] by using 11 equally spaced nodes and constructing Pn(x)."

We will start by building the polynomial *P10* as we have done in many other assignments with *Coef* and *Eval*. Then, we will look at the polynomial against the exact function values, followed by the absolute error plot.

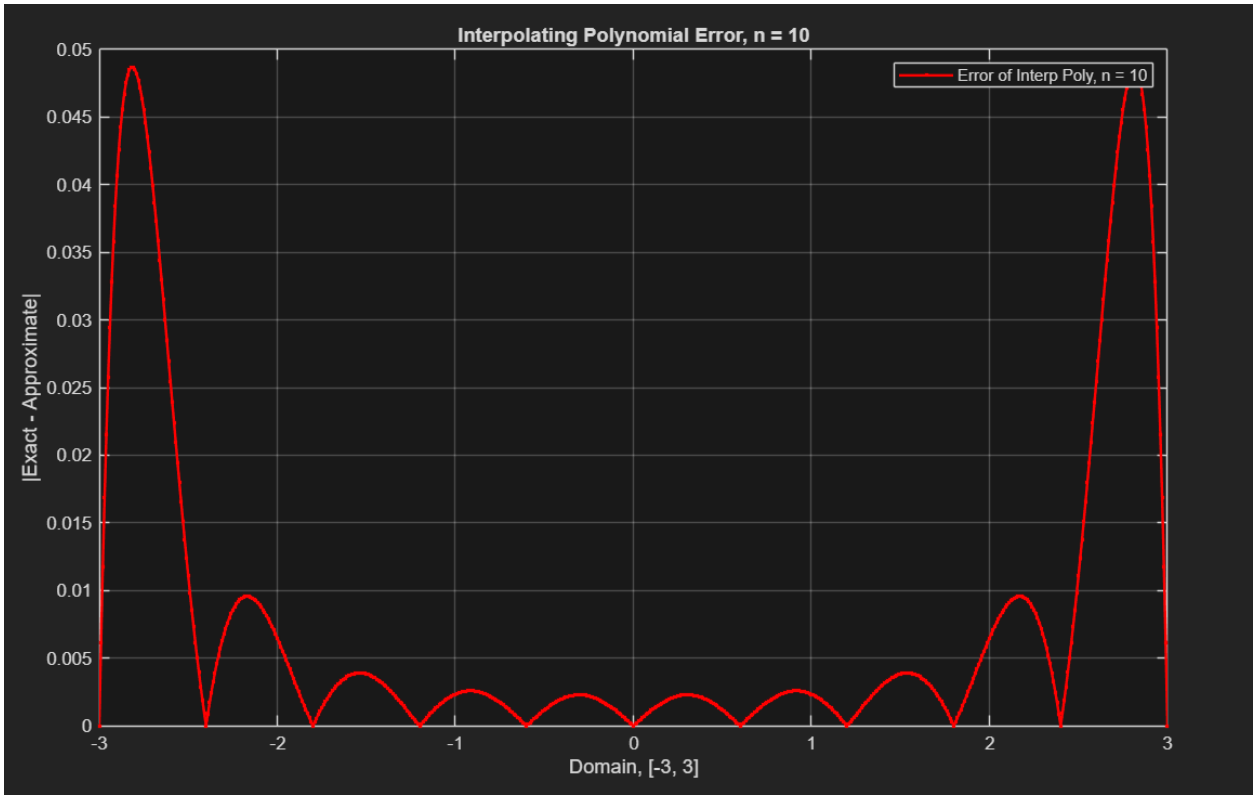
```
%
================================================================
  BUILDING A CLASSIC P10 TO INTERP tanh
%
================================================================

start_pt = -3; end_pt = 3; % initial start and end points

h = 0.01;
% exact function to compare to
x_exact = [start_pt:h:end_pt];
y_exact = tanh(x_exact);

%  BUILD CLASSIC P10 ON [-3, 3]

n = 10; % P10, 11 equispaced nodes (n + 1 nodes)
h_equispaced = (end_pt - start_pt) / n; % get equispace
x_P10 = [start_pt:h_equispaced:end_pt];
y_P10 = tanh(x_P10); % evaluate data points
a_coeff = Coef(n, x_P10, y_P10); % get coefficients

% evaluate the interpolating polynomial at a finer mesh
x_P10_Interp = [start_pt:h:end_pt];
for i = 1:length(x_P10_Interp);
   y_P10_Interp(i) = Eval(n, x_P10, a_coeff, x_P10_Interp(i));
end;

% plot the P10
figure('Name', "P" + n + ", Exact tanh(x)");
plot( ...
   x_exact, y_exact, '--c' , ...
   x_P10_Interp, y_P10_Interp, '--r', ...
   'LineWidth', 1.5 ...
   )
legend("Exact tanh(x) Values", "P" + n);
title("P" + n + " VS Exact tanh(x)");
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Function Output');
% ylim([0, 1]); % limit specified as needed
grid on;
```

```
% error plot
error = abs(y_exact - y_P10_Interp);
figure('Name', 'Absolute Error of Interpolating Polynomial
Approximation'); %plotting the error
plot( ...
   x_exact, error, '.-r', ...
   LineWidth=1.5 ...
);
hold on;
legend( ...
   "Error of Interp Poly, n = " + n ...
   );
title("Interpolating Polynomial Error, n = " + n);
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
grid on;

%
=================================================================
  END CODE FOR COMP PROB 1
%
=================================================================
```

Below is the polynomial plotted against the exact values of *tanh*.



We can already see the notable oscillations occurring–similar to Runge Phenomenon issues. A look at the error plotting makes that even clearer.



We can see above that the error is quite small near the center of the interval, while beginning to diverge wildly at the edges, confirming the oscillation issue. As we have seen in prior assignments, adding more nodes or changing the points used may not necessarily fix this. So, let us try the cubic spline instead, which has a more predictable error guarantee like the degree-1 spline does.

PART B

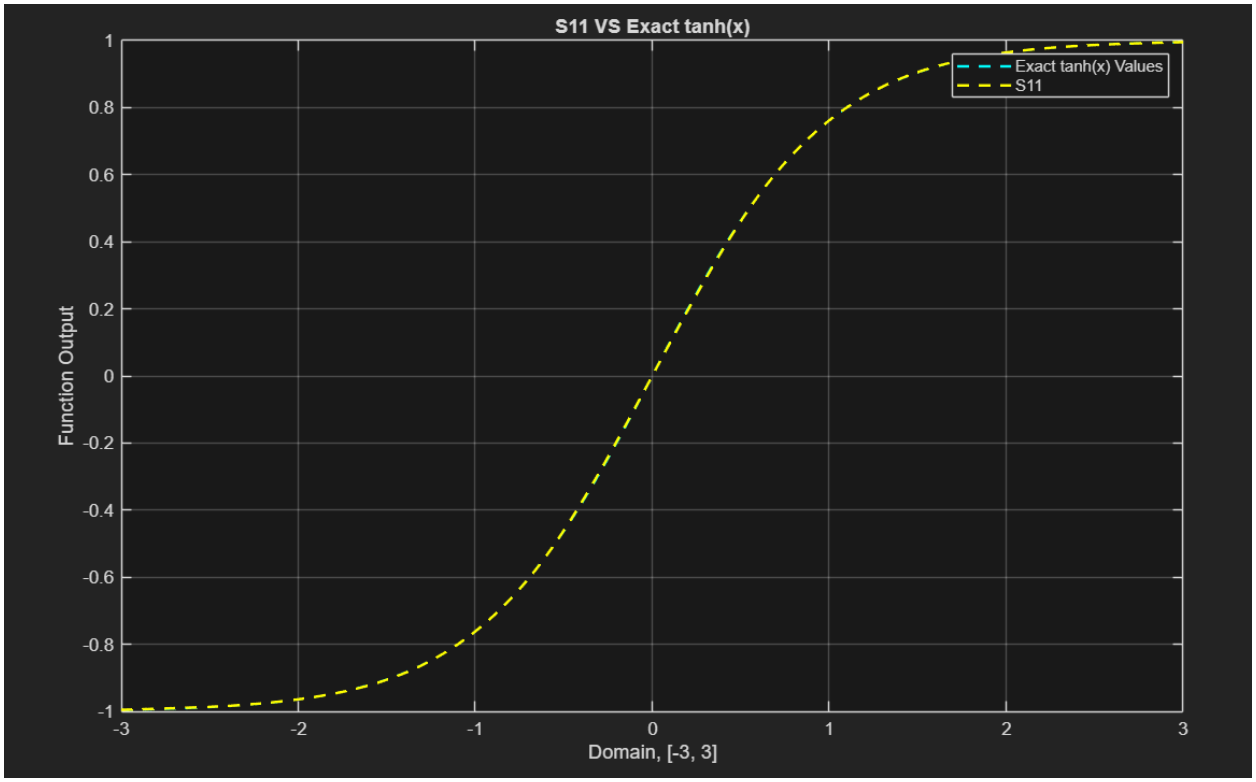
"Then, you can repeat the process and use S(x), cubic splines."

For this part, we will use the same equispaced nodes as *P10*, creating *11* equispaced intervals. Then, we will use the *Spline3_Coef* and *Spline3_Eval* to generate a cubic spline to interpolate the table.

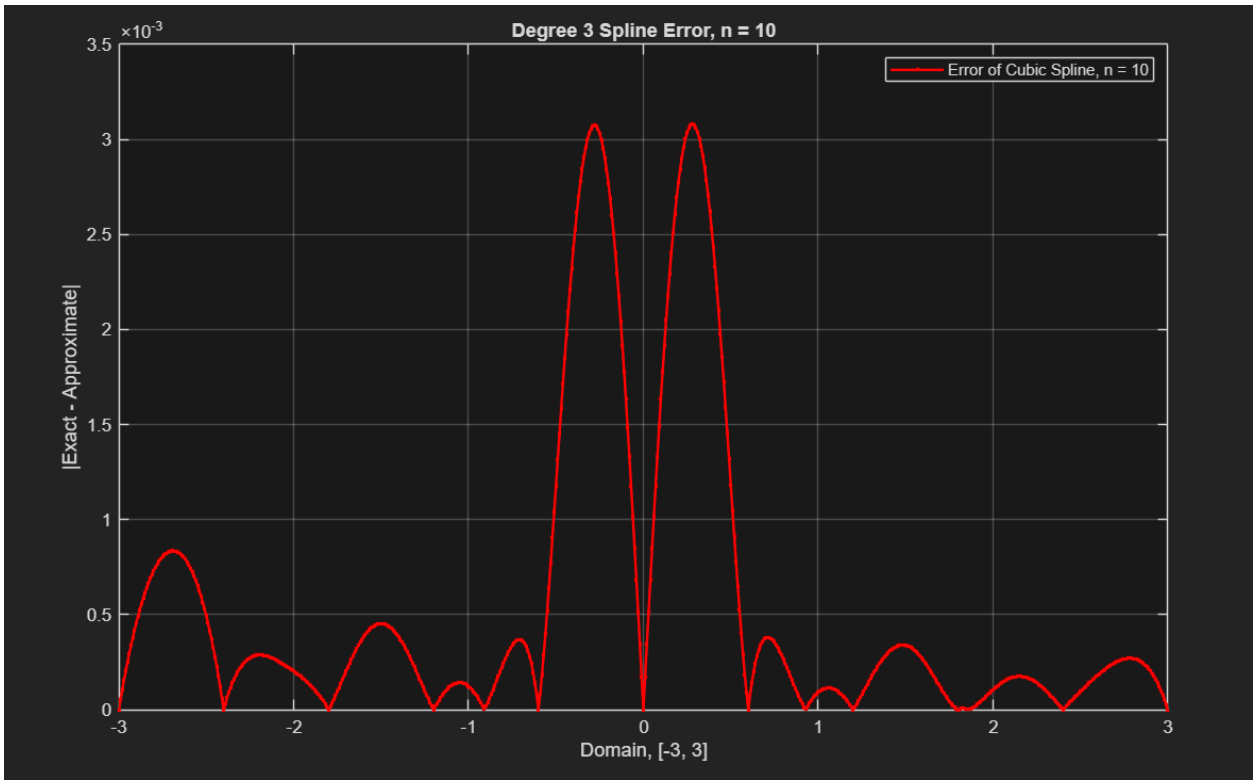
```
%
================================================================
  BUILDING A NATURAL CUBIC SPLINE TO INTERP tanh
%
================================================================

n = 10; % 11 equispaced nodes for now (n + 1 nodes)
h_equispaced = (end_pt - start_pt) / n; % get equispace
x_cubic_spline = [start_pt:h_equispaced:end_pt];
y_cubic_spline = tanh(x_cubic_spline); % evaluate data points

% USING SUPPLIED CODE
x_cubic_interp = [start_pt:h:end_pt];
% gather needed coefficients
[x_cubic_spline, y, z] = Spline3_Coeff(x_cubic_spline,
y_cubic_spline);

% evaluation
for i = 1:length(x_cubic_interp)
   y_cubic_interp(i) = Spline3_Eval(x_cubic_spline, y, z,
x_cubic_interp(i));
end

% plot the cubic spline and exact
figure('Name', "S" + (n + 1) + ", Exact tanh(x)");
plot( ...
```

```
   x_exact, y_exact, '--c' , ...
   x_cubic_interp, y_cubic_interp, '--y', ...
   'LineWidth', 1.5 ...
   )
legend("Exact tanh(x) Values", "S" + (n + 1));
title("S" + (n + 1) + " VS Exact tanh(x)");
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Function Output');
% ylim([0, 1]); % limit specified as needed
grid on;

% error plot
error = abs(y_exact - y_cubic_interp);
figure('Name', 'Absolute Error of Cubic Spline Approximation');
%plotting the error
plot( ...
   x_exact, error, '.-r', ...
   LineWidth=1.5 ...
);
hold on;
legend( ...
   "Error of Cubic Spline, n = " + n ...
   );
title("Degree 3 Spline Error, n = " + n);
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
grid on;

%
================================================================
  END CODE FOR COMP PROB 1
%
================================================================
```

Below is the graphing of the cubic spline *S11* against the exact. As we can already see, the match is phenomenally close to the exact, and we experience none of the oscillations of the polynomial.



Further, let us look at the error.



Some interesting things appear in this error plotting. We notice that we have the opposite trend of the polynomial–the middle of the interval has the highest error margin, along with some slight edge peaking. One hypothesis as to why this might be is due to using cubic functions to approximate a highly non-zero slope linear area. That portion of *tanh* is the most linear portion on that interval, and cubics are naturally curvy. We will see this peaking trend continue through all parts of this project. However, the error is small, so it is likely worth this trade off, especially compared to the polynomial as we will see.

PART C

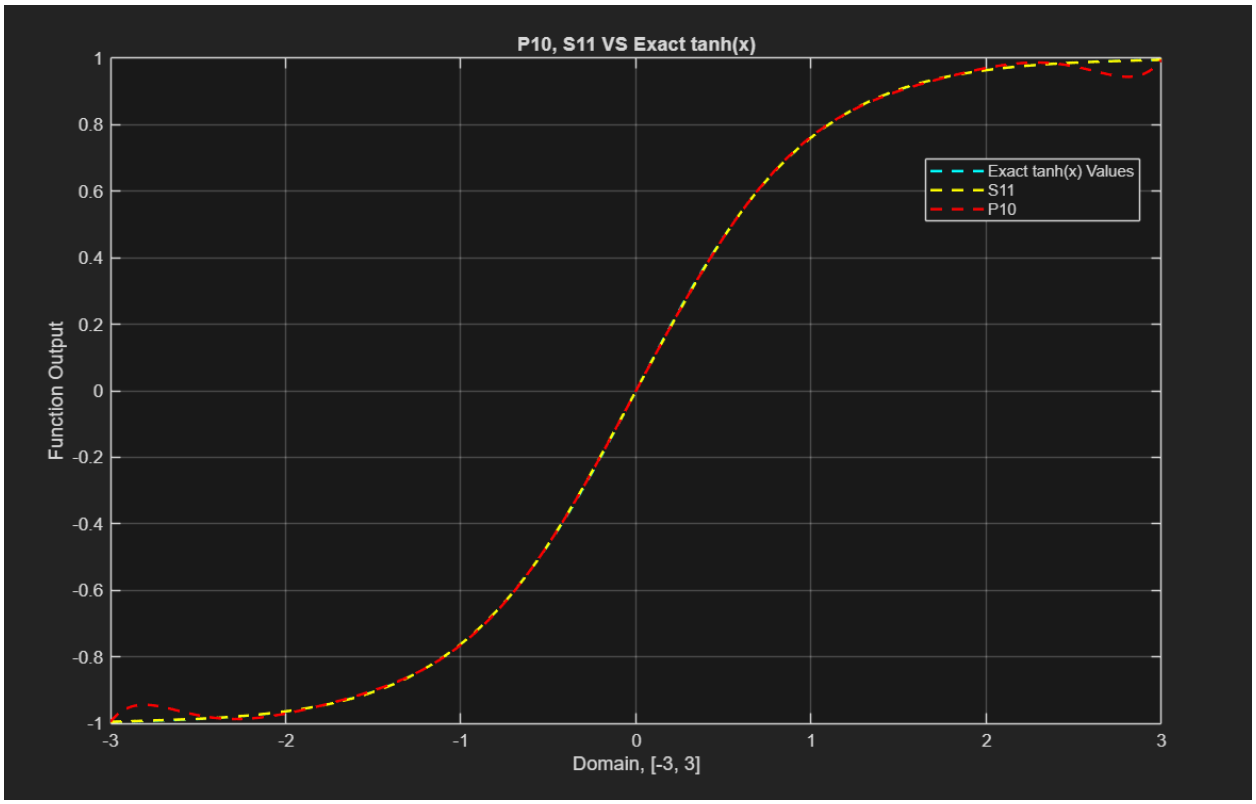
"Plot the original function, the classic interpolating polynomial and the cubic spline in the same figure. Describe your results."

Below is the code to simply plot the two interpolations generated together.

```
%
================================================================
  PLOTTING BOTH CUBIC SPLINE AND P10
%
================================================================

% plot the cubic spline and exact
figure('Name', "P" + n + ", S" + (n + 1) + ", Exact tanh(x)");
plot( ...
   x_exact, y_exact, '--c' , ...
   x_cubic_interp, y_cubic_interp, '--y', ...
   x_P10_Interp, y_P10_Interp, '--r', ...
   'LineWidth', 1.5 ...
   )
legend("Exact tanh(x) Values", "S" + (n + 1), "P" + n);
title("P" + n + ", S" + (n + 1) + " VS Exact tanh(x)");
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Function Output');
% ylim([0, 1]); % limit specified as needed
grid on;

% error plot
error_spline = abs(y_exact - y_cubic_interp);
error_poly = abs(y_exact - y_P10_Interp);
figure('Name', 'Absolute Error of Cubic Spline and Polynomial');
%plotting the error
```

```
plot( ...
    x_exact, error_spline, '.-y', ...
    x_exact, error_poly, '.-r', ...
    LineWidth=1.5 ...
);
hold on;
legend( ...
    "Error of Cubic Spline, n = " + n, ...
    "Error of Interp Poly, n = " + n ...
    );
title("Degree 3 Spline and Polynomial Error, n = " + n);
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
grid on;

%
=========================================================================
  END CODE FOR COMP PROB 1
%
=========================================================================
```

We see both of the interpolations together below, highlighting the differences between them.



Above is the clear difference in approximation–the spline does not experience the clear oscillations of the polynomial. The error of the two plotted together makes this difference far more clear.

The error of the spline is obviously far less than the polynomial (with the admitted slight exception in the middle of the interval). Additionally, the error of the cubic spline trends to *0* exactly where the polynomial fails the most. So, let's look more into where the failure is worse by expanding the intervals.

PART D

"In addition, can you extend your work to [−5, 5] - what happens in this case?"

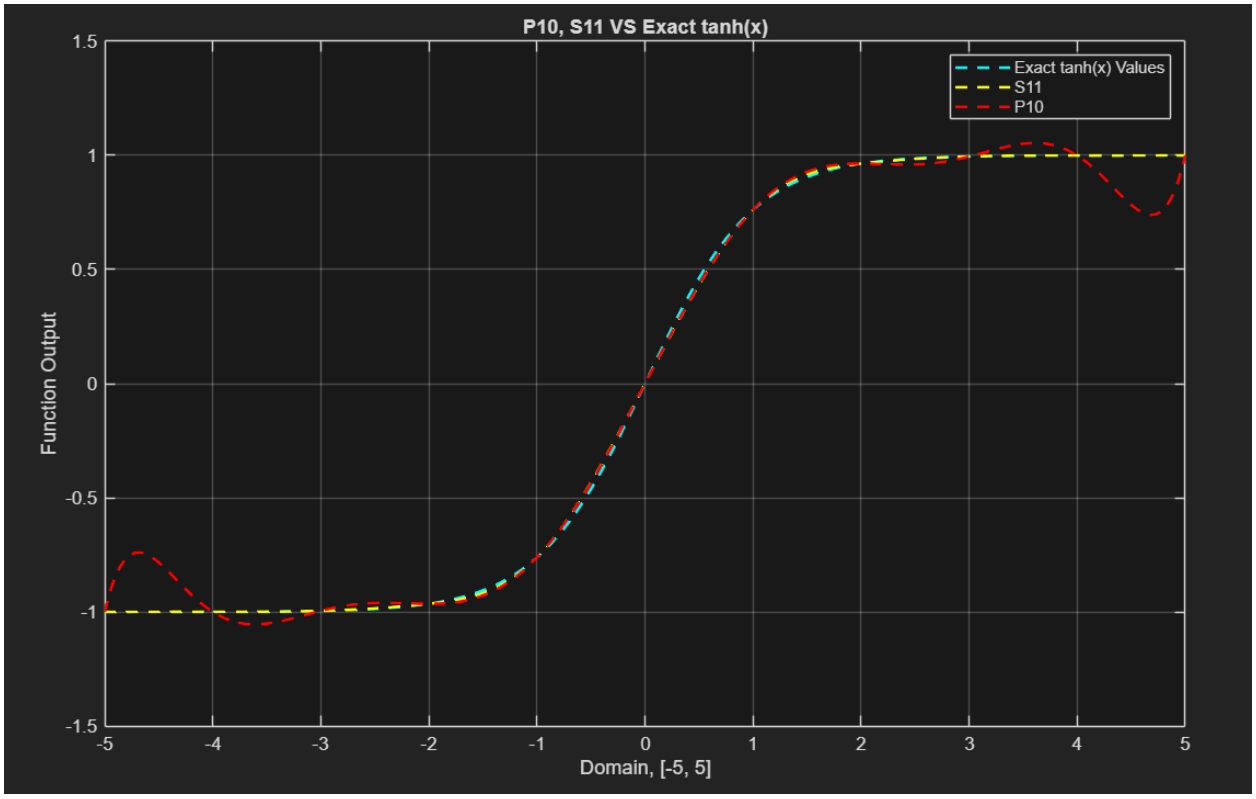
We will change the endpoints and rerun the calculations needed to generate our new interpolations on *[-5, 5]*, again viewing both the normal and error plots.

```
%
============================================================
  EXTENDING WORK TO [-5, 5]
```

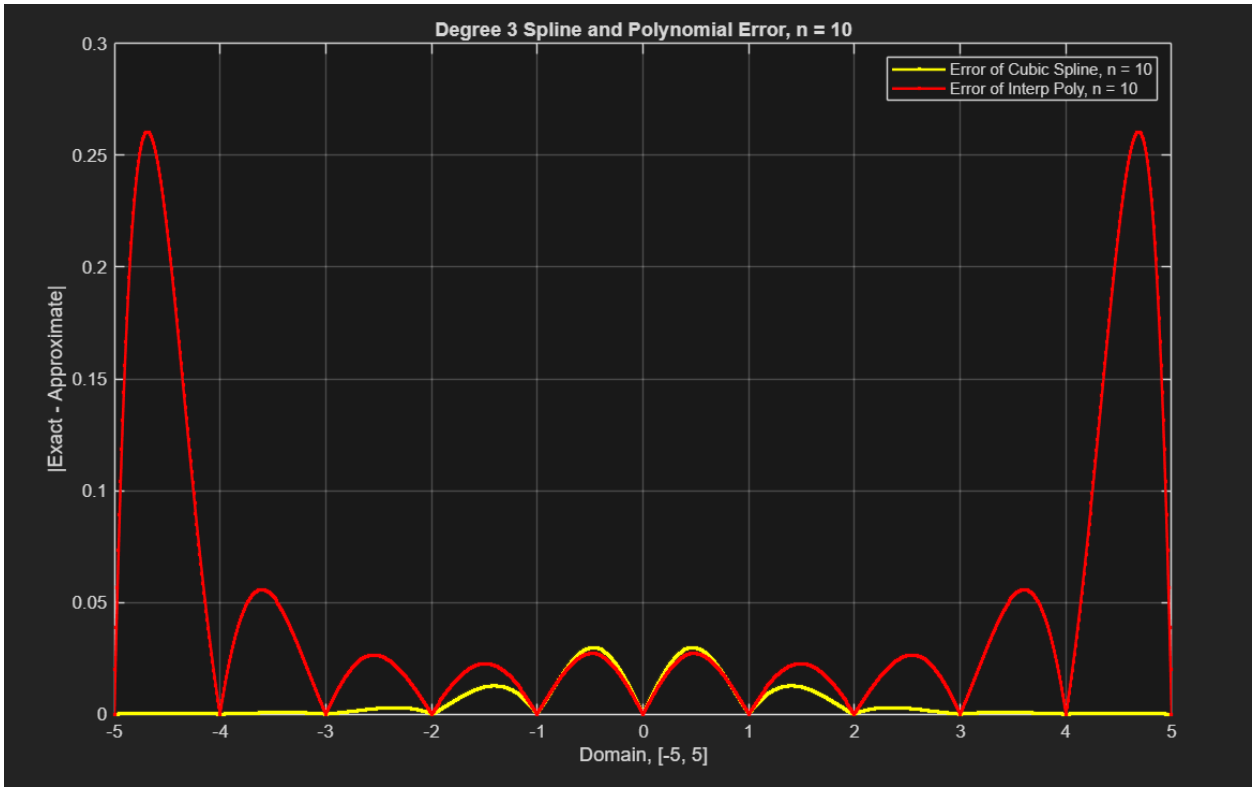
```
%
================================================================

% changed endpoints
start_pt = -5; end_pt = 5;

% exact values
x_exact = [start_pt:h:end_pt];
y_exact = tanh(x_exact);

% polynomial interpolation
h_equispaced = (end_pt - start_pt) / n; % get equispace
x_P10 = [start_pt:h_equispaced:end_pt];
y_P10 = tanh(x_P10); % evaluate data points

a_coeff = Coef(n, x_P10, y_P10); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_P10_Interp = [start_pt:h:end_pt];
for i = 1:length(x_P10_Interp);
   y_P10_Interp(i) = Eval(n, x_P10, a_coeff, x_P10_Interp(i));
end;

% cubic spline interpolation
x_cubic_spline = [start_pt:h_equispaced:end_pt];
y_cubic_spline = tanh(x_cubic_spline); % evaluate data points

% USING SUPPLIED CODE
x_cubic_interp = [start_pt:h:end_pt];

% gather needed coefficients
[x_cubic_spline, y, z] = Spline3_Coeff(x_cubic_spline,
y_cubic_spline);
% evaluation
for i = 1:length(x_cubic_interp)
   y_cubic_interp(i) = Spline3_Eval(x_cubic_spline, y, z,
x_cubic_interp(i));
end

% plot the cubic spline and exact
figure('Name', "P" + n + ", S" + (n + 1) + ", Exact tanh(x),
Changed Interval");
```

```
plot( ...
   x_exact, y_exact, '--c' , ...
   x_cubic_interp, y_cubic_interp, '--y', ...
   x_P10_Interp, y_P10_Interp, '--r', ...
   'LineWidth', 1.5 ...
   )
legend("Exact tanh(x) Values", "S" + (n + 1), "P" + n);
title("P" + n + ", S" + (n + 1) + " VS Exact tanh(x)");
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Function Output');
% ylim([0, 1]); % limit specified as needed
grid on;

% error plot
error_spline = abs(y_exact - y_cubic_interp);
error_poly = abs(y_exact - y_P10_Interp);
figure('Name', 'Absolute Error of Cubic Spline and Polynomial,
Changed Interval'); %plotting the error
plot( ...
   x_exact, error_spline, '.-y', ...
   x_exact, error_poly, '.-r', ...
   LineWidth=1.5 ...
);
hold on;
legend( ...
   "Error of Cubic Spline, n = " + n, ...
   "Error of Interp Poly, n = " + n ...
   );
title("Degree 3 Spline and Polynomial Error, n = " + n);
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
grid on;

%
================================================================
  END CODE FOR COMP PROB 1
%
================================================================
```

We can see from the visual below, we can see that both of the lines are ever so slightly worse at a higher interval with the same number of nodes/knots.



Looking at the error below we can remark upon a couple things.



First, the error in this new interval from both interpolations has increased substantially from the *[-3, 3]* interpolations (a full order for the spline even). However, the spline still has significantly less than the polynomial. Knowing what we know about splines, we can guarantee less error by simply adding more intervals. However, we cannot guarantee that will fix the polynomial in any way. Let us nevertheless investigate the impact of this change on both.

PART E

"Can you continue to increase the number of subintervals to decrease error?"

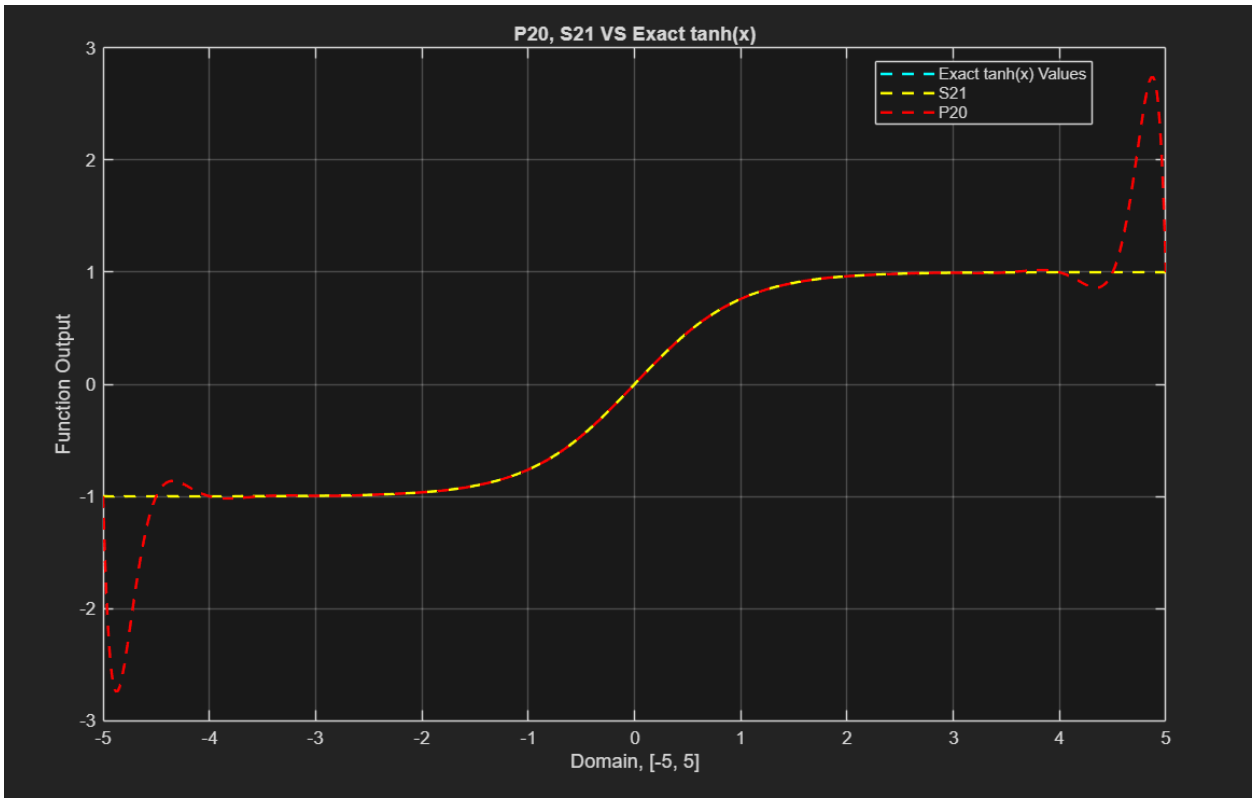
For the following, we will start with increasing *n* to *20* to see the impact.

```
%
================================================================
  INCREASING N
%
================================================================

n = 20; % changed n to see effects on [-5, 5]

% polynomial interpolation
h_equispaced = (end_pt - start_pt) / n; % get equispace
x_Pn = [start_pt:h_equispaced:end_pt];
y_Pn = tanh(x_Pn); % evaluate data points
a_coeff = Coef(n, x_Pn, y_Pn); % get coefficients
% evaluate the interpolating polynomial at a finer mesh
x_Pn_Interp = [start_pt:h:end_pt];
for i = 1:length(x_Pn_Interp);
   y_Pn_Interp(i) = Eval(n, x_Pn, a_coeff, x_Pn_Interp(i));
end;

% cubic spline interpolation
x_cubic_spline = [start_pt:h_equispaced:end_pt];
y_cubic_spline = tanh(x_cubic_spline); % evaluate data points

% USING SUPPLIED CODE
x_cubic_interp = [start_pt:h:end_pt];
% gather needed coefficients
[x_cubic_spline, y, z] = Spline3_Coeff(x_cubic_spline,
y_cubic_spline);
```

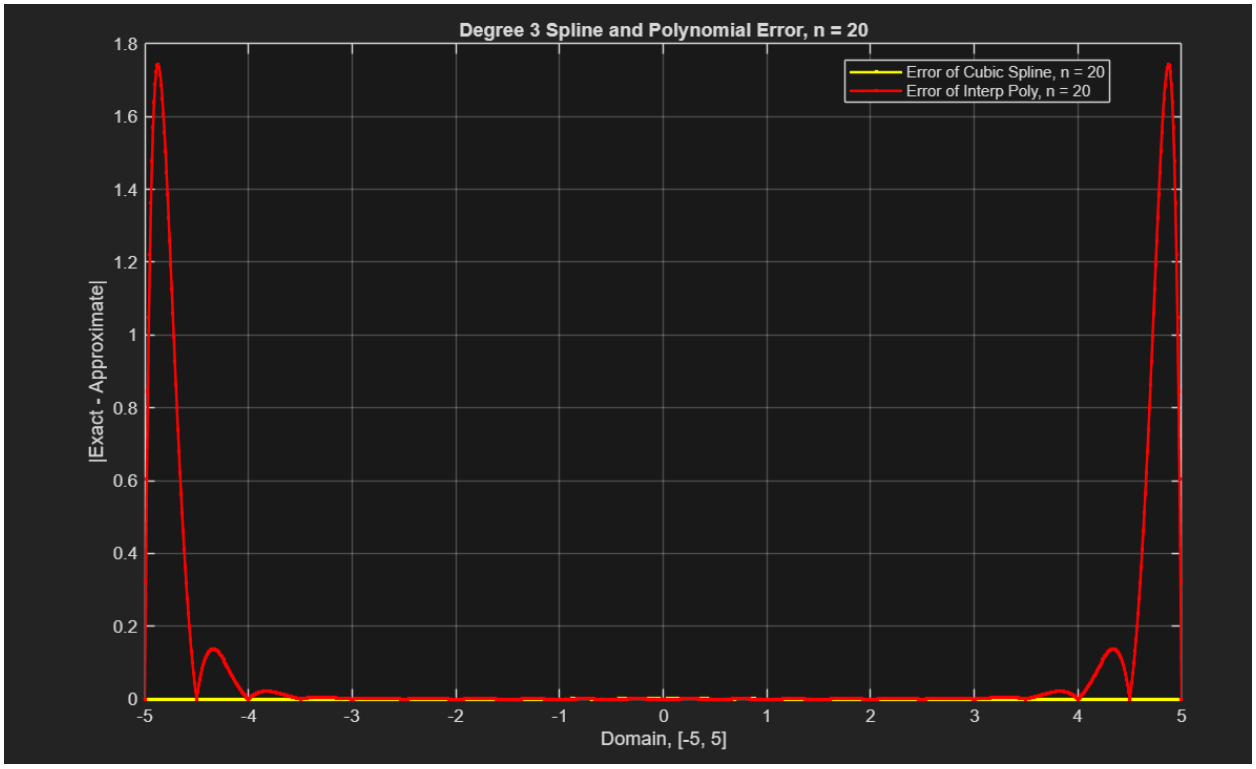
```
% evaluation
for i = 1:length(x_cubic_interp)
   y_cubic_interp(i) = Spline3_Eval(x_cubic_spline, y, z,
x_cubic_interp(i));
end

% plot the cubic spline and exact
figure('Name', "P" + n + ", S" + (n + 1) + ", Exact tanh(x),
Changed n");
plot( ...
   x_exact, y_exact, '--c' , ...
   x_cubic_interp, y_cubic_interp, '--y', ...
   x_Pn_Interp, y_Pn_Interp, '--r', ...
   'LineWidth', 1.5 ...
   )
legend("Exact tanh(x) Values", "S" + (n + 1), "P" + n);
title("P" + n + ", S" + (n + 1) + " VS Exact tanh(x)");
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('Function Output');
% ylim([0, 1]); % limit specified as needed
grid on;

% error plot
error_spline = abs(y_exact - y_cubic_interp);
error_poly = abs(y_exact - y_Pn_Interp);
figure('Name', 'Absolute Error of Cubic Spline and Polynomial,
Changed n'); %plotting the error
plot( ...
   x_exact, error_spline, '.-y', ...
   x_exact, error_poly, '.-r', ...
   LineWidth=1.5 ...
);
hold on;
legend( ...
   "Error of Cubic Spline, n = " + n, ...
   "Error of Interp Poly, n = " + n ...
   );
title("Degree 3 Spline and Polynomial Error, n = " + n);
xlabel("Domain, [" + start_pt + ", " + end_pt + "]");
ylabel('|Exact - Approximate|');
% this limit will get you zoomed into the worst of the cubic
```

```
% ylim([0, 0.002]);
grid on;

%
==========================================================================
  END CODE FOR COMP PROB 1
%
==========================================================================
```

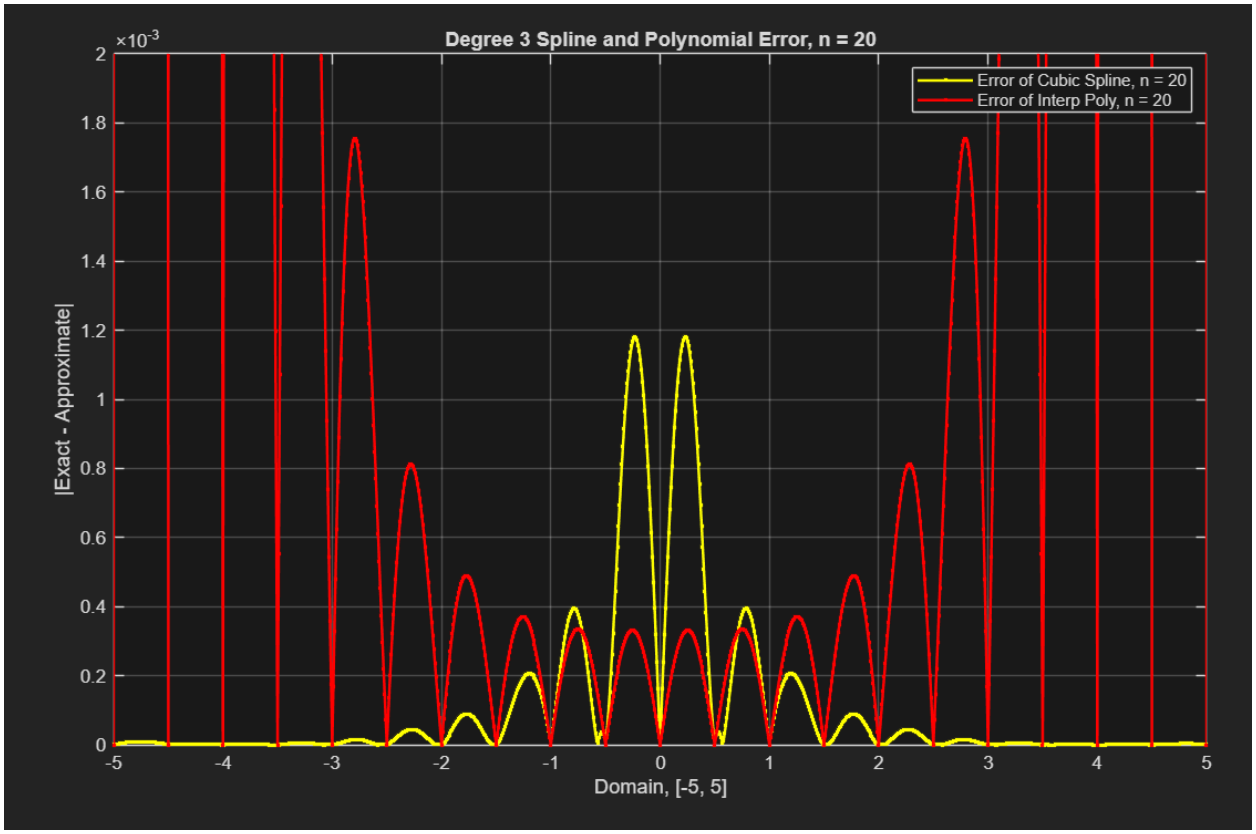
As we expect, the spline improves, and the polynomial begins to oscillate more.



And, the error along with to highlight the difference.

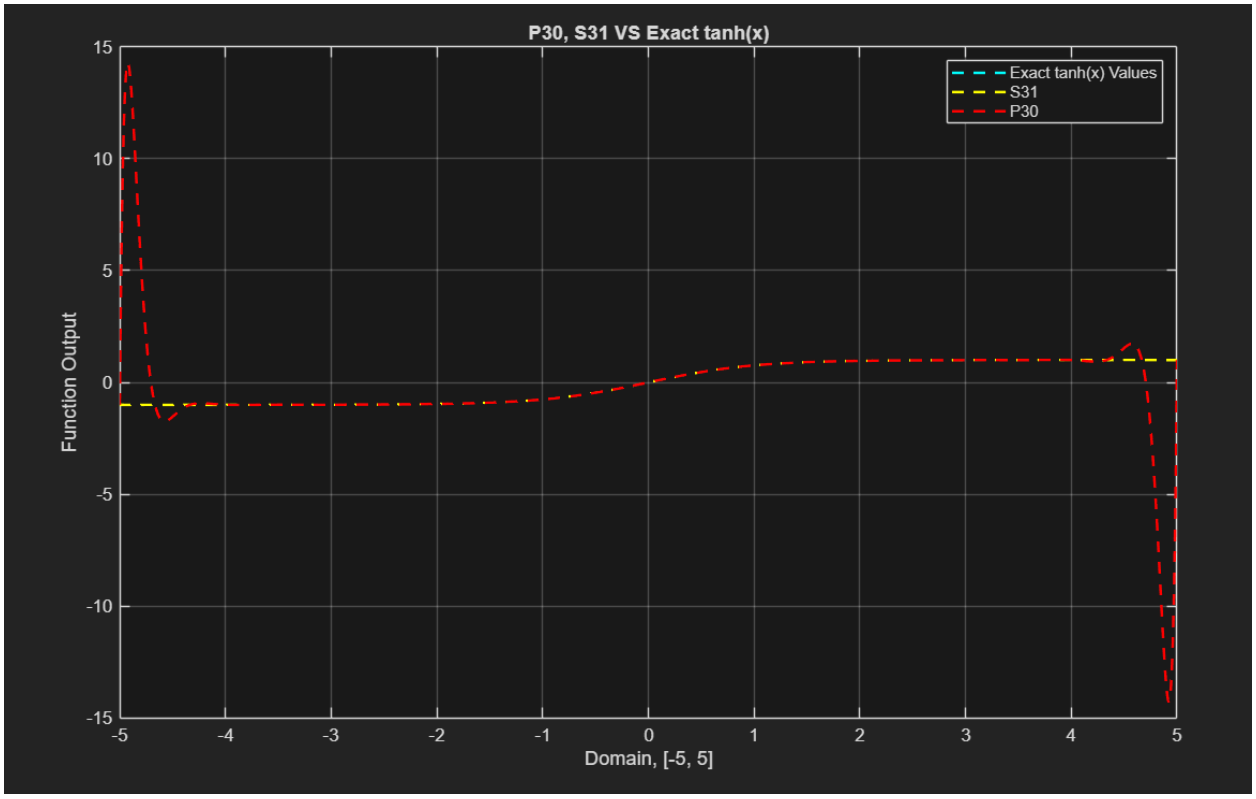


We can see that the polynomial error starts to explode quickly around the edges, where the spline succeeds in decreasing dramatically. We can also zoom in on the spline error to see this impact.

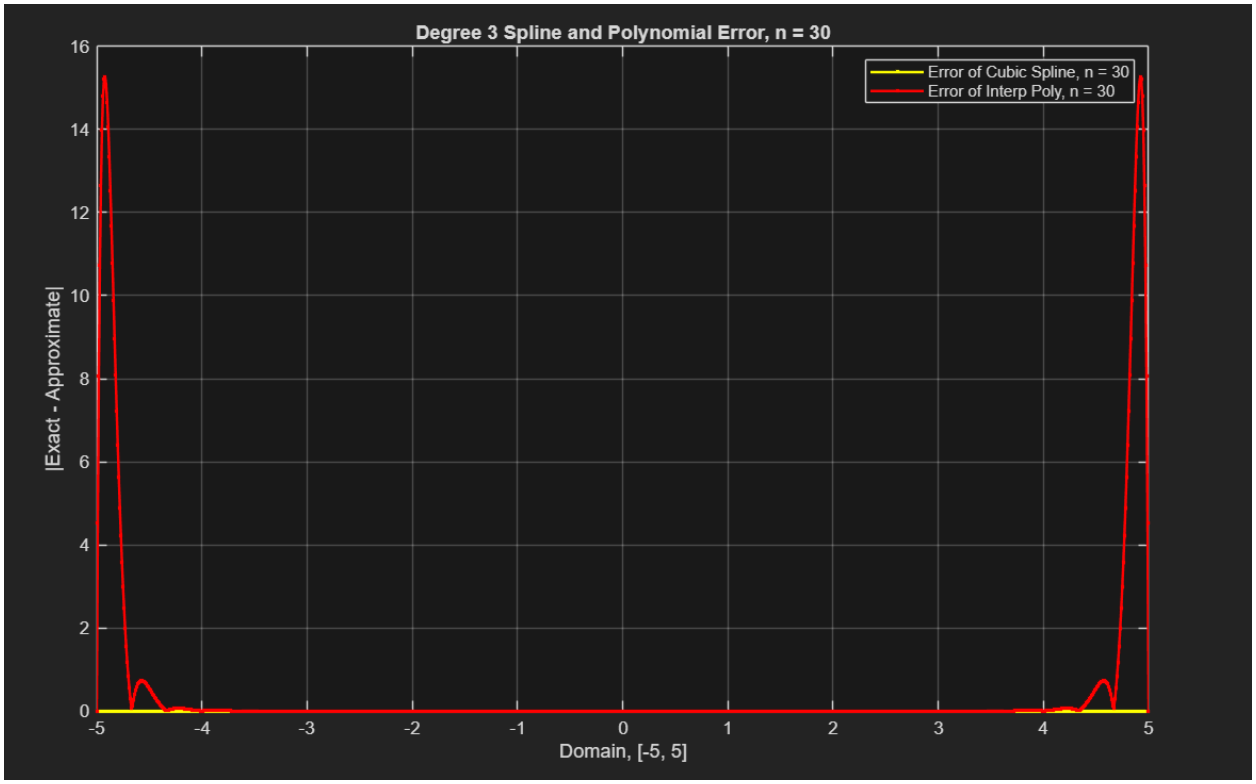


The error decrease in the spline was by an entire order of magnitude. Admittedly, the spline still performs worse than the polynomial in the middle, but overall approximates far better than the polynomial, and behaves far more predictably in terms of improvement.

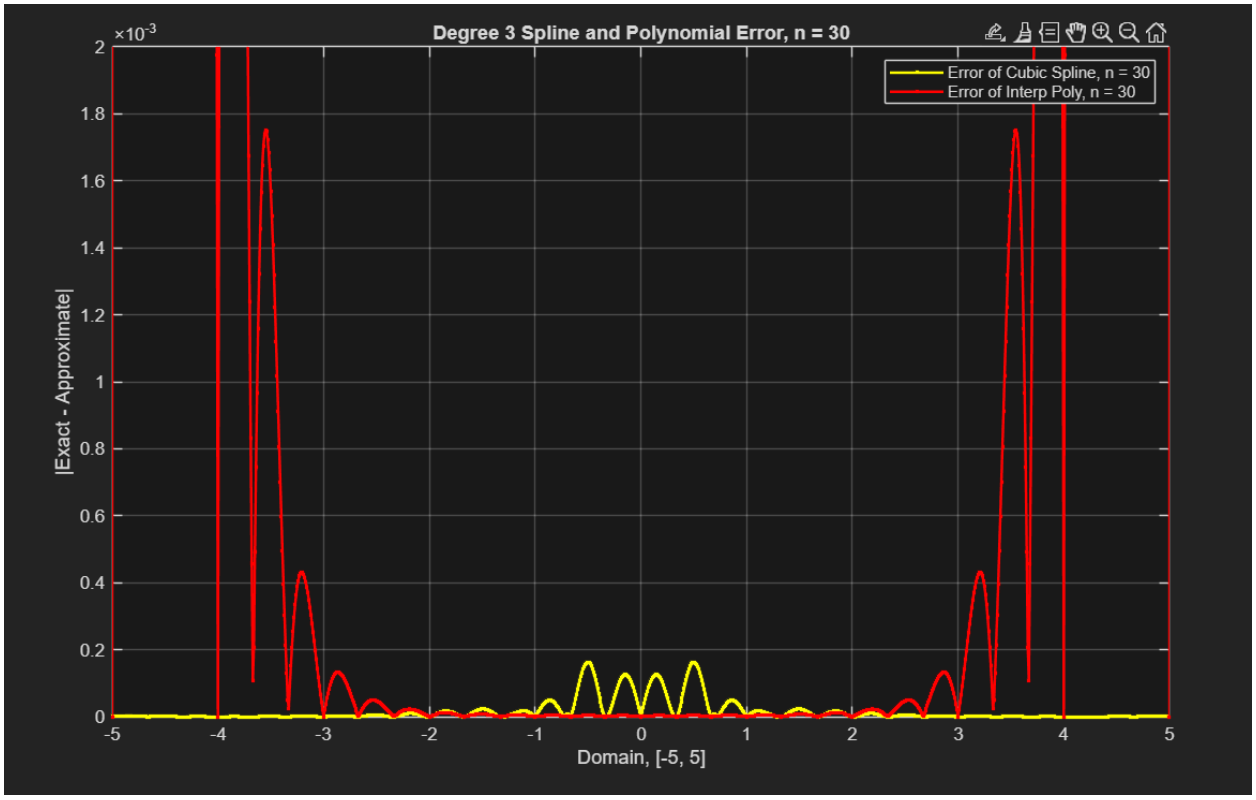
As a sanity check, let's briefly change *n* to *30* to see that the effect continues.



As we can see, the polynomial is oscillating at the edges almost 5 times as much as when $n = 20$. And the error comparison likewise confirms the trend.



The horrible peak is evident on the edges, and shrinking the range to zoom into the spline highlights the issues yet again.



Clearly, the spline is the better choice, predictably dropping in error by yet another order. Although it fails more in specific spots, it is absolutely overall the better choice in accuracy. This is especially true considering that the *n* can always continue being increased to consistently lessen the inaccuracy of the spline.

CONCLUSION

In conclusion, polynomial interpolation is a valid way to interpolate functions and data as needed. However, the use is arguable when there is such a wild potential for error in specific situations. The cubic spline is mathematically more complex, perhaps, but the tradeoff is likely worth any extra computation and implementation in serious use cases. Although not every situation may need a cubic spline–situations in which it may be overkill–it is a powerful and reliable tool to produce a highly accurate interpolation.